Assignment 1

CSA0903-Java Programming

Name: Sowmiya. D

Reg No.:192511063

Scenario — Campus Smart-Parking System

The university campus has a multi-level parking facility used by students, staff and visitors. The facility has several floors, and each floor has a fixed number of numbered slots. Vehicles entering the campus may be two-wheelers, cars or electric vehicles (EVs). EVs may also use charging slots, which are charged separately.

When a vehicle enters, the system must allocate a suitable free slot, issue a parking ticket with a unique ticket number, and record the entry time. When the vehicle leaves, the system must release the slot and compute the parking fee. Fees depend on the vehicle type and the duration of parking. Staff and students holding a valid monthly pass park free of charge, while visitors pay the full fee.

The facility may contain:

1 floor × 20 slots → 3 floors × 50 slots → 5 floors × 100 slots

You are required to analyse the problem, design an object-oriented model for it, implement the model in Java and validate it. Your design must remain correct and easy to extend as the facility and its rules grow.

Design Challenge:

The university now wants to add new vehicle categories and a new pricing rule without rewriting the classes that already work.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 2 – Choose the Data Representation 3 Marks

Bloom's Level: K4 – Analyse

Decide how the data of the system will be represented using Java data types and arrays.

Discuss:

- The data type chosen for each field (slot number, vehicle number, entry time, duration, fee) and why
- How the floors and slots will be represented (one-dimensional, two-dimensional or jagged array) and why
- Where type conversion or casting is required (for example: converting minutes to hours for billing) and the risk of losing precision
- How you will read input (Scanner or command-line arguments)

Show at least one example where **integer division or an incorrect cast would produce a wrong fee**, and how your design avoids it.

Alternative Concept and Complexity Analysis

Instead of a jagged array String[floor][slot], suppose the slots are stored in a single onedimensional array using index = floor × slotsPerFloor + slot, or in an ArrayList<String[]>. What would change in your solution when floors have different numbers of slots? Analyse the time to locate a slot and the space required in each representation.

Evidence Expected

Justified choice of data types and array structure, with an identified casting risk and its resolution.
TASK 02 – Choose the Data Representation

1. Selection of Java Data Types

Field	**Data type**	**Reason**
Floor number	int	Stores floor index
Slot number	int	Stores slot index
Vehicle number	String	Stores registration number
Entry time	LocalDateTime	Stores date and time
Exit time	LocalDateTime	Stores exit date and time
Duration	long	Stores parking duration in minutes
Parking fee	double	Stores decimal fee values
Vehicle type	String or enum	Identifies vehicle category
Pass validity	boolean	Checks whether the pass is valid

2. Representation of Floors and Slots

I choose a jagged two-dimensional array to represent the parking facility.

```
String[][] slots = {
    new String[20],
    new String[50],
    new String[100]
};
```

- Each row represents one parking floor.
- Each column represents a parking slot.
- null indicates an available slot.

- A vehicle registration number indicates an occupied slot.

Reason: A jagged array allows different floors to have different numbers of parking slots.

3. Type Conversion and Casting

Type conversion is required when calculating parking duration and fees.

For example:

- Minutes are converted into hours.
- Integer values are converted into decimal values during fee calculation.

Incorrect calculation:

int minutes = 90;

double hours = minutes / 60;

Output: 1.0

This happens because integer division removes the decimal part.

Correct calculation:

int minutes = 90;

double hours = minutes / 60.0;

Output: 1.5

Using 60.0 ensures floating-point division and avoids precision loss from integer division.

4. Reading Input

I choose the Scanner class to read user input because it is simple and suitable for a console-based Java application.

Scanner sc = new Scanner(System.in);

System.out.print("Enter vehicle number: ");

String regNo = sc.nextLine();

System.out.print("Enter vehicle type: ");

String type = sc.nextLine();

5. Alternative Concept and Complexity Analysis

Alternative 1: One-Dimensional Array

String[] slots = new String[150];

The index is calculated using:

index = floor × slotsPerFloor + slot

This works easily when every floor has an equal number of slots. When floors have different slot counts, the calculation requires maintaining the starting index of each floor.

Alternative 2: ArrayList

ArrayList<String[]> floors = new ArrayList<>();

Each element stores one floor's slot array. It supports adding floors dynamically.

6. Comparison

Representation	**Slot access**	**Space complexity**
Jagged array	O(1)	O(S)
1D array	O(1)	O(F × S)
ArrayList of arrays	O(1)	O(S)

Here, F represents the number of floors and S represents the total number of parking slots.

Conclusion

I choose a jagged two-dimensional array because different floors may contain different numbers of parking slots. It provides direct slot access, efficient memory usage and flexibility when the parking facility is expanded.